

Assembly

Embedding assembly in C →
(1) Interact directly with the hardware

(2) Performance optimization (relevant only for rare hardware)

Two ways of embedding assembly

1) Basic asm

2) Extended asm

Basic asm: → __asm__ ("movq \$3, %rax");

(1) Opcodes differ based on the type of operands (movq, movl, addq, addl)

(2) Sequence of operands is reversed, destination is at the end.

GAS (GNU assembler)

Basic assembly is something where you are not specifying anything except assembly code. Why should you avoid basic assembly?

```
int x = 2;
__asm__ ("movq $3, %rax");
printf("%d", x);
```

Extended assembly →

```
__asm__ [① volatile] [② goto]
(Assembler Template ③
: [Output Operands] ④)
```

: [Input Operands] ⑤
 : [Clobbers] ⑥
 ⑦: [Goto Labels] ;

① volatile: → Instruction reordering helps optimize execution and is used by compilers. The compiler should not reorder this (embedded assembly) instruction if volatile is specified

② goto: → Specify to the compiler that there is a jump instruction.

③ Assembler Template: → Specify the operation, but without using the registers

"addl %1, %2"

add rax, 10
addl \$10, %rax

④ Output Operand: → "r"(a) "m"(a)
 r → register
 m → memory

Writing to a register: $\rightarrow$ "=" modifier
 $\underbrace{["r" \text{ or } "=m"]}_{\text{Valid}}$

Both read and write: "+" modifier

⑤ Input Operands: $\rightarrow$ Same syntax as output operands.

```
int main(void) {  
    int sum, x, y;  
    x = 5;  
    y = 10;
```

// sum = x + y;

```
--asm-- ("addq, %1, %2" : "=r"(sum)  
          : "r"(x), "0"(y));
```

```
printf("%d", sum);
```

"0": It is a constraint which asks the compiler ~~to~~ to map the two variables into the same register
b mapped into ¹ $\rightarrow$ a and some reg.

Exercise: $\rightarrow$ Copy the content of one variable into another.

```
"addq %1 %2 |t|n  
movq %2. %3"
```

Two or more instructions can be given.

⑥ Clobbers: $\rightarrow$ Specify to the assembler that a specific list ~~of~~ of memory locations or registers would be changed in assembly.

Clobbers: $\rightarrow$ %rax, %rbx

⑦ Goto Labels: $\rightarrow$ Specify the list of labels to which you might jump from assembly.

ARTIX, UBUNTU, LINUX, UNIX,
GNU ✓

LINUX: → A specific kernel used by most systems today

UNIX: → Traditional OS which is present since 1969. → AT&T System V, BSD, Berkeley UNIX, IBM → AIX

Linux was designed to be compatible with other UNIX systems..

There is a standardization system, which gives the basic options or facilities that all these OS should provide. This standard is called POSIX. Linux is fully POSIX-compliant.
MacOSX ,, ,, ,, ,

Linux kernel is open-source → You can download and change things in the kernel.

What ^{are} ~~is~~ Artix, Ubuntu, Fedora,etc?

→ Distributions of Linux

⇓

They Collect the source code of kernel and System utilities, build them and give them to the users in a more usable form.

GNU: → Organize whose aim was to design open-source software so that people could use them more conveniently.

Compiler, GUI tools, text editors, etc.

But they were not able to build a proper kernel.

Linux was used together with the GNU Tools to make one complete OS.

GNU/Linux.

glibc
Android → Own GUI, its ~~being~~ ^{virtual} environment is called Android Runtime, only the Linux kernel is still same.

Are installing and building a distribution same or different?

→ Installing implies just copying the binaries that have been built possibly on some other machine.

→ Building implies compiling and linking the software.

How are you building the software for embedded systems?

→ Cross-compilation is necessary where you specify a flag to indicate that it is being built for a different ISA.